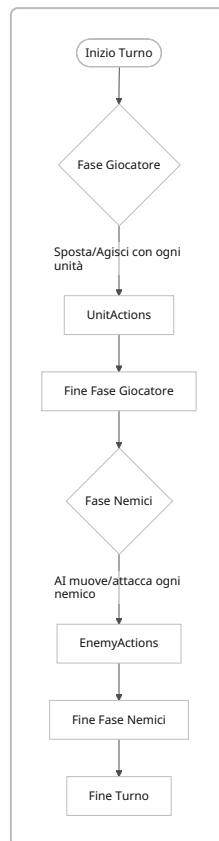


Executive Summary

- **Terreno variegato:** Includere tipi come copertura (muri, cespugli), terreno difficile (fango, acqua) e ostacoli (trincee, barriere) che influenzino movimento e visibilità 【10†L99-L107】 . Ad esempio, i boschi rallentano e forniscono copertura, l'altura migliora l'accuratezza e l'acqua rallenta drasticamente (tabella 1). Evitare inconsistenze nei cover system: un crate deve sempre comportarsi uguale 【17†L289-L294】 . Meccaniche dinamiche (es. distruggere coperture o incendiare terreno) aggiungono profondità, ma vanno usate con coerenza o eliminate 【6†L138-L145】 【10†L99-L107】 .
- **Personaggi e classi:** Definire ruoli classici (Tank, DPS corpo a corpo, DPS a distanza, Support/Healer, CC) con statistiche (HP, difesa, risorse) e risorse (es. punti azione, mana) differenziate 【19†L54-L62】 【19†L67-L75】 . Progressione tramite livelli, equipaggiamento e talent tree (scelte permanenti) fornisce senso di crescita. Ad esempio, i talenti di WoW permettono a un Druido di cambiare ruolo 【19†L130-L137】 . Queste scelte aumentano la profondità, ma richiedono attento bilanciamento. Tabella 2 riassume punti di forza e debolezze dei ruoli principali.
- **Azioni e abilità:** Le azioni includono **movimento** (uso di PA per spostarsi), **attacco base** (singolo bersaglio, costo basso) e **abilità speciali** (es. magie, abilità uniche) o **interazioni ambientali** (spingere massi, aprire porte). Ogni abilità ha parametri come costo (PA, cooldown), raggio di azione (singolo bersaglio o area) e tempo di esecuzione. Esempi concreti in tabella 3 (nomi, danno base, costo PA, CD, area). Le abilità possono scalare con statistiche (es. +% danno per punto di forza). È importante testare tempi e costi per evitare turni troppo lunghi. Unreal Engine supporta sistemi data-driven (Gameplay Ability System) per definire queste abilità via dati (Data Asset/CSV) 【33†L23-L32】 .
- **Interazioni Azione-Terreno-Personaggio:** Le azioni possono colpire personaggi o modificare terreno. Ad esempio, un attacco a proiettile infuoca il suolo (terreno → condizione). Il terreno può fornire copertura (+difesa) o ostacolare (difficile da attraversare) 【10†L99-L107】 . Condizioni e trigger (es. stordito, avvelenato) permettono combo: spingere un nemico nel fuoco crea ingaggio ambientale. I buff/debuff possono dipendere dal terreno (es. +Danno se il nemico è bagnato). È utile consentire manovre come knockback per creare effetti a catena 【6†L112-L119】 .



- Fasi di gioco e condizioni:** Un **turno** può essere strutturato a team (turno intero del giocatore, poi dei nemici) o a unità singole. Tra i turni si aggiornano stati come recupero PA, durata buff e trigger di eventi ambientali (per es. pioggia che spegne incendi). Stati di campo (es. nebbia, notte) possono alterare visibilità o abilità. Condizioni speciali (avvelenamento, paralisi) influenzano temporaneamente unità. In certi giochi i round contengono più di un giro (es. in *Fire Emblem* tutti i giocatori, poi tutti i nemici) **[15†L117-L124]** . Diagramma sopra mostra un flusso tipico di turno.
- Personalizzazione:** Differenziare estetica (skin) da funzionalità. Per la **custom di gameplay**, usare loadout (equipaggiamento e abilità selezionate prima della partita) e talent tree per sbloccare modifiche permanenti (incrementi di stat o abilità speciali). Esempi: gemme/incantesimi inseribili nelle armi per effetti speciali (fire arrow, effetto acid) **[6†L118-L124]** , oppure rune che aggiungono effetti (knockback, cura extra). La scelta tra molte opzioni accresce replayability, ma richiede un'interfaccia chiara. Un albero di talenti consente specializzazioni (es. Tank più difensivo o più agro **[19†L130-L137]**). Varianti: abilitare moduli (oggetti equipaggiabili con passivi), sistemi di crafting. *Pro:* grande varietà, senso di crescita. *Contro:* complessità e possibile squilibrio se non testato.
- Ruoli e bilanciamento:** Classi archetipo includono **Tank** (alto HP/difesa, attacco basso), **DPS melee** (alto danno fisico, mobilità media), **DPS ranged** (danno a distanza, bassa difesa), **Healer/Support** (cura/bonus, offensiva debole), **CC/Utility** (stordimenti, buff/debuff, danno ridotto). Vedi tabella 2 per pro/contro. Ogni ruolo dovrebbe avere contromisure: per esempio, tank protegge il team ma è vulnerabile alle debolezze elementali; gli attacchi ad area danno vantaggio contro gruppi ma poco contro tank. Il sistema trinità (Tank/Healer/DPS) è base **[19†L54-L62]** , ma è utile mixare ruoli (es. un DPS con qualche cura **[19†L67-L75]**). Bilanciare genera e risorse (PA/mana) garantisce che nessun ruolo domini sempre.

Ruolo	Punti di forza	Debolezze
Tank	Molta salute, alta difesa, agro (aggro)	Danno basso, mobili più lenti
DPS Cc	Alto danno in mischia, buona mobilità	Difesa bassa, range limitato
DPS Ranged	Alto danno a distanza, flessibilità	Richiede copertura, meno resistenza
Support/Healer	Cura e buff alleati	Danno quasi nullo, risorse critiche
Controllo (CC)	Abilità di stordire/imbavagliare	Danno medio-basso, dipende da tempistica

• **Esempi di abilità:** Alcuni esempi concreti (tutte modificabili per bilanciamento):

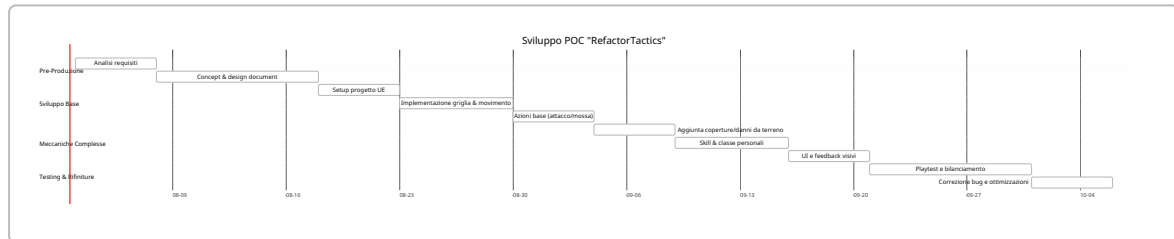
Abilità	Tipo	Danno/ Effetto	Raggio/ AOE	Costo PA	Cooldown	Trigger
Colpo Trascinante	Attacco	40 danni + spinta	1 casella	1	0	Knockback se bordo (es.)
Freccia Infuocata	Magia	30 danni + +5 DOT/turno	Raggio 5, AOE 2	2	2 turni	Ignite terreno al punto d'impatto
Scudo Benedetto	Supporto	+20 Difesa a un alleato	0 (contatto)	1	3 turni	Se alleato <50% HP
Terremoto del Guerriero	AoE/ Danno	25 danni + stordimento	Raggio 3, area 2	2	4 turni	-

(Esempio: "Freccia Infuocata" infligge danno magico e innesca un bonus di danno nel tempo se il bersaglio è nel terreno infuocato. "Scudo Benedetto" si autoattiva quando un alleato scende sotto il 50% di salute.)

• **Meccaniche emergenti e design pattern:** Fondamentale prevedere *counterplay* e *risk/reward*. Per esempio, abilità forti dovrebbero comportare un rischio (danno all'utilizzatore o lunga ricarica), creando tensione decisionale. Il concetto di hard/soft counter da picchiaduro [37†L288-L299] si traduce in tattica come mosse che "cancellano" certe strategie nemiche (es. parata che annulla un colpo forte) o reazioni condizionate (es. schivata con tempismo). Varietà nei contatori arricchisce il gioco [37†L288-L299]. Evitare info overload: troppo detailing (stati, buff) rende difficile orientarsi [26†L100-L107]. Il gioco dovrebbe permettere decisioni sensate senza paralisi da opzioni, facilitando momenti di "brillantezza tattica" senza doverlo essere ad ogni mossa [26†L119-L127].

• **Telemetria e metriche POC:** Raccogliere dati di uso durante il testing: percentuale di utilizzo di ogni abilità, tempo medio per turno, tassi di uccisione e morti per ruolo, copertura delle mappe utilizzata. Monitorare esempi come abilità mai usate (da ribilanciare) o unità sempre in vantaggio. Metriche consigliate: *Action Usage* (quante volte e quali azioni vengono scelte), *Damage per Action*, *Success Rate* (hit percentuali), *Turn Length*, distribuzione di vittorie/sconfitte per composizione. Questi dati aiutano a bilanciare costi e danni ed evidenziare meccaniche in crisi.

- **Implementazione rapida in UE:** Sfruttare un sistema data-driven. Unreal offre il Gameplay Ability System (GAS) con AttributeSets e Gameplay Effects **【33†L23-L32】** . Definire ogni abilità come Data Asset (o tabella CSV) contenente parametri (costo PA, danno, area, cooldown) e codificare in Blueprint/C++ uno “executor” che applica quegli effetti ai bersagli. Le **DataTables** (CSV) in UE possono contenere l'elenco delle abilità con i valori di base. Ad esempio, un file JSON/CSV può specificare “danno=50, raggio=3, effetto=Stordimento”. Le Interfacce Blueprint permettono di chiamare dinamicamente questi dati. In breve, **Attribute** per stat (HP, forza, resistenza), **Ability** per logiche (es. AttackTask, BuffTask), e **GameplayEffect** per modifiche (a valori) riducono codice rigido **【33†L23-L32】** .



Fase	Attività Principali	Stima Ore (min)	Stima Ore (max)	Deliverable
Pre-produzione	Analisi/Concept/Game Design Document	20h	40h	Documento GDD completo
Impostazione progetto	Creazione progetto UE5, asset liberi (ad esempio da Kenney e Quaternius)	10h	20h	Progetto iniziale funzionante
Movimenti e turni	Implementazione griglia, spostamenti e azioni base	20h	35h	Meccanica di base (grid + turni)
Terreno e coperture	Aggiunta tipi di terreno, cover, effetti ambientali	15h	25h	Mappe e ostacoli funzionanti
Personaggi/skill di base	Definizione classi, abilità fondamentali	25h	40h	Sistema classi e abilità POC
Iterazione e debug	Playtest, bilanciamento abilità, correzioni	15h	30h	POC bilanciato, bugfix

Le stime basse/alte dipendono dalla complessità dei sistemi (es. **Low** = meccaniche semplici, asset di prototipo, **High** = implementazione completa di talent tree e effetti avanzati).

Fonti: Analisi basata su linee guida di game design (es. uso di coperture e terreno **【10†L99-L107】** , coerenza delle meccaniche **【17†L289-L294】** **【6†L138-L145】** , ruoli classici **【19†L54-L62】** **【19†L67-L75】** **【15†L117-L124】** , sistemi data-driven **【33†L23-L32】** , strategie di counterplay **【37†L288-L299】**).